

```
In [140... import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
```

```
In [115... DATA='../..data/brca.csv'
```

```
In [116... df = pd.read_csv(DATA)

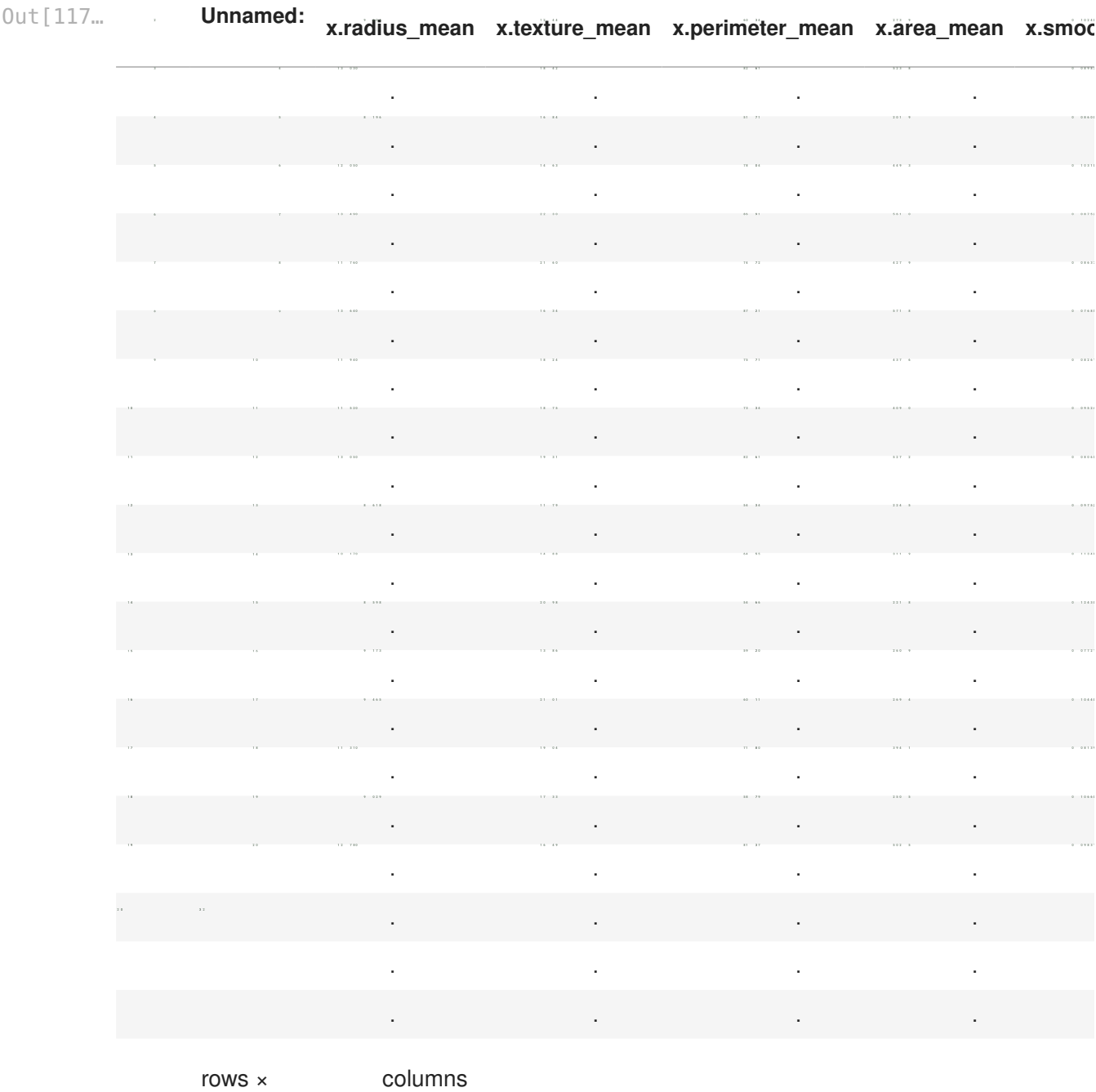
df.head()
```

Out[116]:

Unnamed: 0

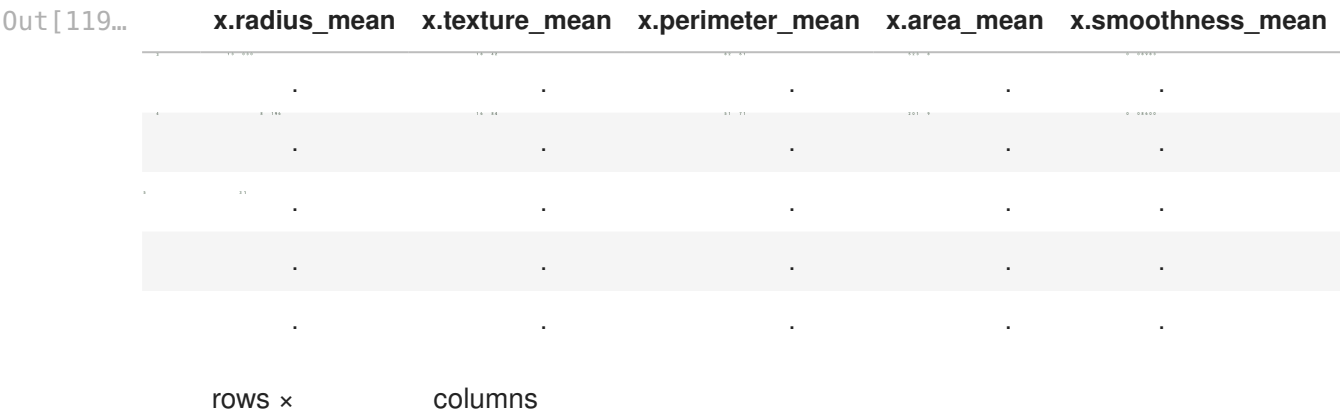
	x.radius_mean	x.texture_mean	x.perimeter_mean	x.area_mean	x.smooth
0	12.000	16.000	40.000	270.0	0.10000
1	12.000	16.000	40.000	270.0	0.10000
2	12.000	16.000	40.000	270.0	0.10000
3	12.000	16.000	40.000	270.0	0.10000
4	12.000	16.000	40.000	270.0	0.10000
5	12.000	16.000	40.000	270.0	0.10000
6	12.000	16.000	40.000	270.0	0.10000
7	12.000	16.000	40.000	270.0	0.10000
8	12.000	16.000	40.000	270.0	0.10000
9	12.000	16.000	40.000	270.0	0.10000
10	12.000	16.000	40.000	270.0	0.10000
11	12.000	16.000	40.000	270.0	0.10000
12	12.000	16.000	40.000	270.0	0.10000
13	12.000	16.000	40.000	270.0	0.10000
14	12.000	16.000	40.000	270.0	0.10000
15	12.000	16.000	40.000	270.0	0.10000
16	12.000	16.000	40.000	270.0	0.10000
17	12.000	16.000	40.000	270.0	0.10000
18	12.000	16.000	40.000	270.0	0.10000
19	12.000	16.000	40.000	270.0	0.10000
20	12.000	16.000	40.000	270.0	0.10000
21	12.000	16.000	40.000	270.0	0.10000
22	12.000	16.000	40.000	270.0	0.10000
23	12.000	16.000	40.000	270.0	0.10000
24	12.000	16.000	40.000	270.0	0.10000
25	12.000	16.000	40.000	270.0	0.10000
26	12.000	16.000	40.000	270.0	0.10000
27	12.000	16.000	40.000	270.0	0.10000
28	12.000	16.000	40.000	270.0	0.10000
29	12.000	16.000	40.000	270.0	0.10000
30	12.000	16.000	40.000	270.0	0.10000
31	12.000	16.000	40.000	270.0	0.10000
32	12.000	16.000	40.000	270.0	0.10000
33	12.000	16.000	40.000	270.0	0.10000
34	12.000	16.000	40.000	270.0	0.10000
35	12.000	16.000	40.000	270.0	0.10000
36	12.000	16.000	40.000	270.0	0.10000
37	12.000	16.000	40.000	270.0	0.10000
38	12.000	16.000	40.000	270.0	0.10000
39	12.000	16.000	40.000	270.0	0.10000
40	12.000	16.000	40.000	270.0	0.10000
41	12.000	16.000	40.000	270.0	0.10000
42	12.000	16.000	40.000	270.0	0.10000
43	12.000	16.000	40.000	270.0	0.10000
44	12.000	16.000	40.000	270.0	0.10000
45	12.000	16.000	40.000	270.0	0.10000
46	12.000	16.000	40.000	270.0	0.10000
47	12.000	16.000	40.000	270.0	0.10000
48	12.000	16.000	40.000	270.0	0.10000
49	12.000	16.000	40.000	270.0	0.10000
50	12.000	16.000	40.000	270.0	0.10000
51	12.000	16.000	40.000	270.0	0.10000
52	12.000	16.000	40.000	270.0	0.10000
53	12.000	16.000	40.000	270.0	0.10000
54	12.000	16.000	40.000	270.0	0.10000
55	12.000	16.000	40.000	270.0	0.10000
56	12.000	16.000	40.000	270.0	0.10000
57	12.000	16.000	40.000	270.0	0.10000
58	12.000	16.000	40.000	270.0	0.10000
59	12.000	16.000	40.000	270.0	0.10000
60	12.000	16.000	40.000	270.0	0.10000
61					

```
In [117... df.head(20)
```



```
In [118... df.drop(columns='Unnamed: 0', inplace=True)
```

```
In [119... df.head()
```



```
In [120... summary = df.describe()  
print(summary)
```

	x.radius_mean	x.texture_mean	x.perimeter_mean	x.area_mean	\
count	569.000000	569.000000	569.000000	569.000000	
mean	14.127292	19.289649	91.969033	654.889104	
std	3.524049	4.301036	24.298981	351.914129	
min	6.981000	9.710000	43.790000	143.500000	
25%	11.700000	16.170000	75.170000	420.300000	
50%	13.370000	18.840000	86.240000	551.100000	
75%	15.780000	21.800000	104.100000	782.700000	
max	28.110000	39.280000	188.500000	2501.000000	

	x.smoothness_mean	x.compactness_mean	x.concavity_mean	\
count	569.000000	569.000000	569.000000	
mean	0.096360	0.104341	0.088799	
std	0.014064	0.052813	0.079720	
min	0.052630	0.019380	0.000000	
25%	0.086370	0.064920	0.029560	
50%	0.095870	0.092630	0.061540	
75%	0.105300	0.130400	0.130700	
max	0.163400	0.345400	0.426800	

	x.concave_pts_mean	x.symmetry_mean	x.fractal_dim_mean	...	\
count	569.000000	569.000000	569.000000	...	
mean	0.048919	0.181162	0.062798	...	
std	0.038803	0.027414	0.007060	...	
min	0.000000	0.106000	0.049960	...	
25%	0.020310	0.161900	0.057700	...	
50%	0.033500	0.179200	0.061540	...	
75%	0.074000	0.195700	0.066120	...	
max	0.201200	0.304000	0.097440	...	

	x.radius_worst	x.texture_worst	x.perimeter_worst	x.area_worst	\
count	569.000000	569.000000	569.000000	569.000000	
mean	16.269190	25.677223	107.261213	880.583128	
std	4.833242	6.146258	33.602542	569.356993	
min	7.930000	12.020000	50.410000	185.200000	
25%	13.010000	21.080000	84.110000	515.300000	
50%	14.970000	25.410000	97.660000	686.500000	
75%	18.790000	29.720000	125.400000	1084.000000	
max	36.040000	49.540000	251.200000	4254.000000	

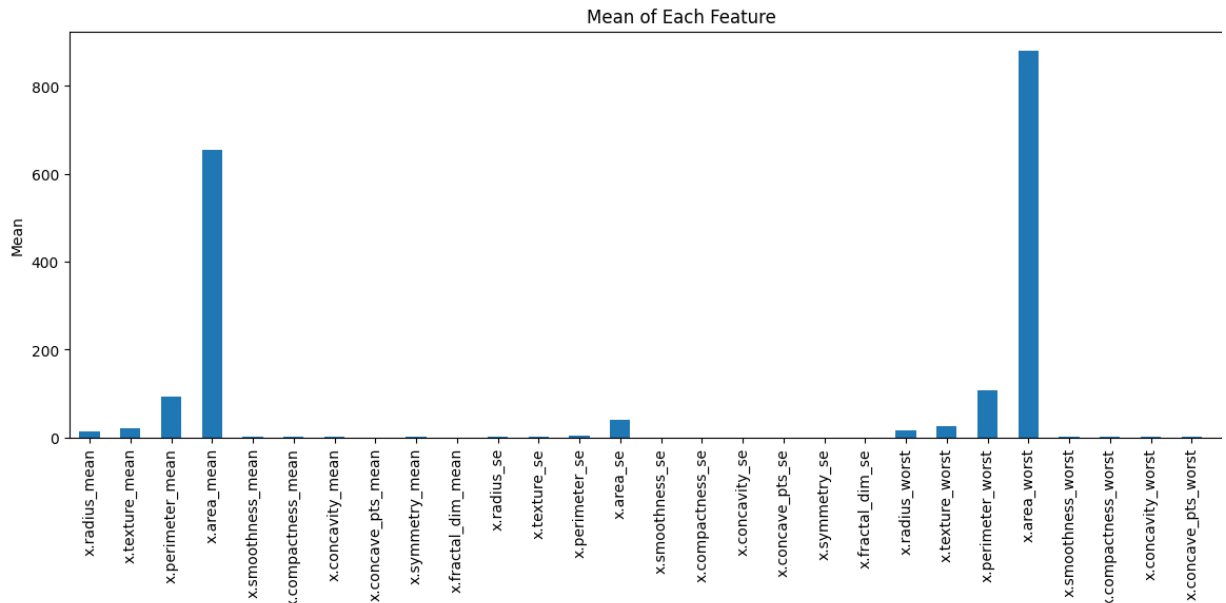
	x.smoothness_worst	x.compactness_worst	x.concavity_worst	\
count	569.000000	569.000000	569.000000	
mean	0.132369	0.254265	0.272188	
std	0.022832	0.157336	0.208624	
min	0.071170	0.027290	0.000000	
25%	0.116600	0.147200	0.114500	
50%	0.131300	0.211900	0.226700	
75%	0.146000	0.339100	0.382900	
max	0.222600	1.058000	1.252000	

	x.concave_pts_worst	x.symmetry_worst	x.fractal_dim_worst
count	569.000000	569.000000	569.000000
mean	0.114606	0.290076	0.083946
std	0.065732	0.061867	0.018061
min	0.000000	0.156500	0.055040
25%	0.064930	0.250400	0.071460
50%	0.099930	0.282200	0.080040
75%	0.161400	0.317900	0.092080
max	0.291000	0.663800	0.207500

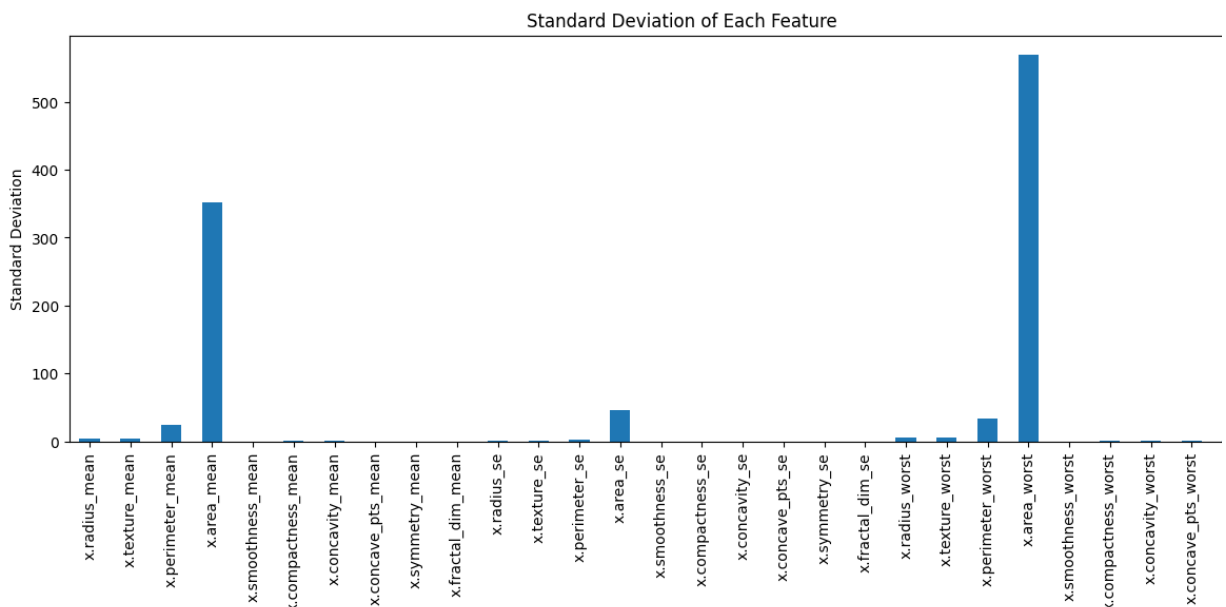
[8 rows x 30 columns]

```
In [121... def display_plots(column, k, title, lable):
    summary.loc[column].plot(kind=k, figsize=(15,5))
    plt.title(title)
    plt.ylabel(lable)
    plt.xticks(rotation=90)
    plt.show()
```

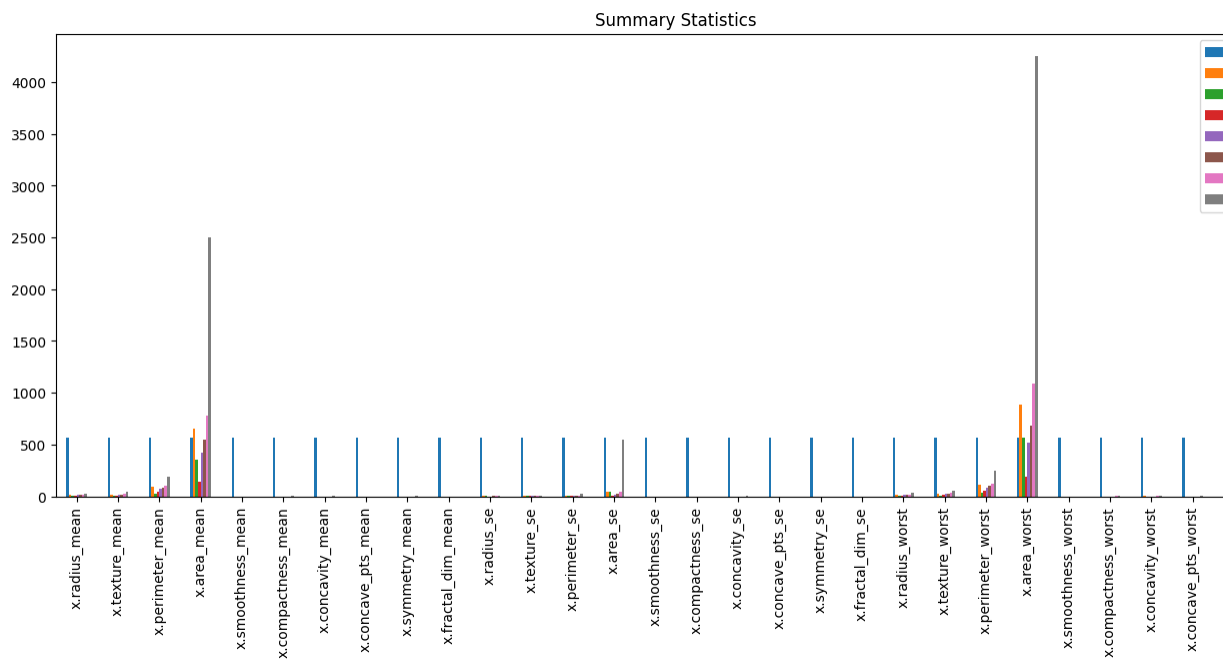
```
In [122... display_plots("mean", "bar", "Mean of Each Feature", "Mean")
```



```
In [123... display_plots("std", "bar", "Standard Deviation of Each Feature", "Standard Deviation")
```



```
In [124... summary.T.plot(kind="bar", figsize=(16,6))
plt.title("Summary Statistics")
plt.xticks(rotation=90)
plt.show()
```



```
In [125... from sklearn.preprocessing import StandardScaler

# Separate features and target
X = df.drop("y", axis=1)
y = df["y"]

# Create scaler
scaler = StandardScaler()

# Scale all features
X_scaled = scaler.fit_transform(X)

# Convert back to DataFrame
X_scaled = pd.DataFrame(X_scaled, columns=X.columns)
y = y.map({
    "B": 1,
    "M": 0
})

print(X_scaled.head())
```

	x.radius_mean	x.texture_mean	x.perimeter_mean	x.area_mean	\
0	-0.166799	-1.147162	-0.185728	-0.251957	
1	-0.297446	-0.833008	-0.261106	-0.383638	
2	-1.313080	-1.593959	-1.302806	-1.083572	
3	-0.311646	-0.202373	-0.385500	-0.372831	
4	-1.684571	-0.570050	-1.658278	-1.288347	

	x.smoothness_mean	x.compactness_mean	x.concavity_mean	\
0	0.101747	-0.436850	-0.278210	
1	0.792763	0.429422	-0.541362	
2	0.429819	-0.747086	-0.743748	
3	-0.464730	-1.263703	-0.793214	
4	-0.737294	-0.851130	-0.915500	

	x.concave_pts_mean	x.symmetry_mean	x.fractal_dim_mean	...	\
0	-0.028609	0.267911	-0.728310	...	
1	-0.459627	0.567289	0.753087	...	
2	-0.726337	0.012345	0.886341	...	
3	-0.507861	-1.258183	-0.590802	...	
4	-1.109197	-0.155598	0.316465	...	

	x.radius_worst	x.texture_worst	x.perimeter_worst	x.area_worst	\
0	-0.240048	-1.045005	-0.225217	-0.297761	
1	-0.366368	-0.844707	-0.332744	-0.439624	
2	-1.250611	-1.631243	-1.254913	-0.994422	
3	-0.614867	-0.466909	-0.679153	-0.588344	
4	-1.512777	-0.605327	-1.489328	-1.122222	

	x.smoothness_worst	x.compactness_worst	x.concavity_worst	\
0	0.509873	-0.489605	-0.159223	
1	-0.051226	0.148443	-0.399099	
2	0.001377	-0.887193	-0.880434	
3	-1.549975	-1.323648	-1.073966	
4	-0.116980	-0.754239	-0.975761	

	x.concave_pts_worst	x.symmetry_worst	x.fractal_dim_worst
0	0.216123	0.123347	-0.629292
1	-0.636110	0.458227	-0.117250
2	-0.796903	-0.729224	-0.344455
3	-0.981753	-1.478256	-1.233324
4	-1.354653	0.330422	-0.546168

[5 rows x 30 columns]

```
In [126... summary = X_scaled.describe()
print(summary)
```

	x.radius_mean	x.texture_mean	x.perimeter_mean	x.area_mean	\
count	5.690000e+02	5.690000e+02	5.690000e+02	5.690000e+02	
mean	-1.998011e-16	-7.492542e-16	-9.990056e-17	-1.998011e-16	
std	1.000880e+00	1.000880e+00	1.000880e+00	1.000880e+00	
min	-2.029648e+00	-2.229249e+00	-1.984504e+00	-1.454443e+00	
25%	-6.893853e-01	-7.259631e-01	-6.919555e-01	-6.671955e-01	
50%	-2.150816e-01	-1.046362e-01	-2.359800e-01	-2.951869e-01	
75%	4.693926e-01	5.841756e-01	4.996769e-01	3.635073e-01	
max	3.971288e+00	4.651889e+00	3.976130e+00	5.250529e+00	

	x.smoothness_mean	x.compactness_mean	x.concavity_mean	\
count	5.690000e+02	5.690000e+02	569.000000	
mean	1.498508e-16	1.998011e-16	0.000000	
std	1.000880e+00	1.000880e+00	1.000880	
min	-3.112085e+00	-1.610136e+00	-1.114873	
25%	-7.109628e-01	-7.470860e-01	-0.743748	
50%	-3.489108e-02	-2.219405e-01	-0.342240	
75%	6.361990e-01	4.938569e-01	0.526062	
max	4.770911e+00	4.568425e+00	4.243589	

	x.concave_pts_mean	x.symmetry_mean	x.fractal_dim_mean	...	\
count	5.690000e+02	5.690000e+02	5.690000e+02	...	
mean	9.990056e-17	1.498508e-16	4.745277e-16	...	
std	1.000880e+00	1.000880e+00	1.000880e+00	...	
min	-1.261820e+00	-2.744117e+00	-1.819865e+00	...	
25%	-7.379438e-01	-7.032397e-01	-7.226392e-01	...	
50%	-3.977212e-01	-7.162650e-02	-1.782793e-01	...	
75%	6.469351e-01	5.307792e-01	4.709834e-01	...	
max	3.927930e+00	4.484751e+00	4.910919e+00	...	

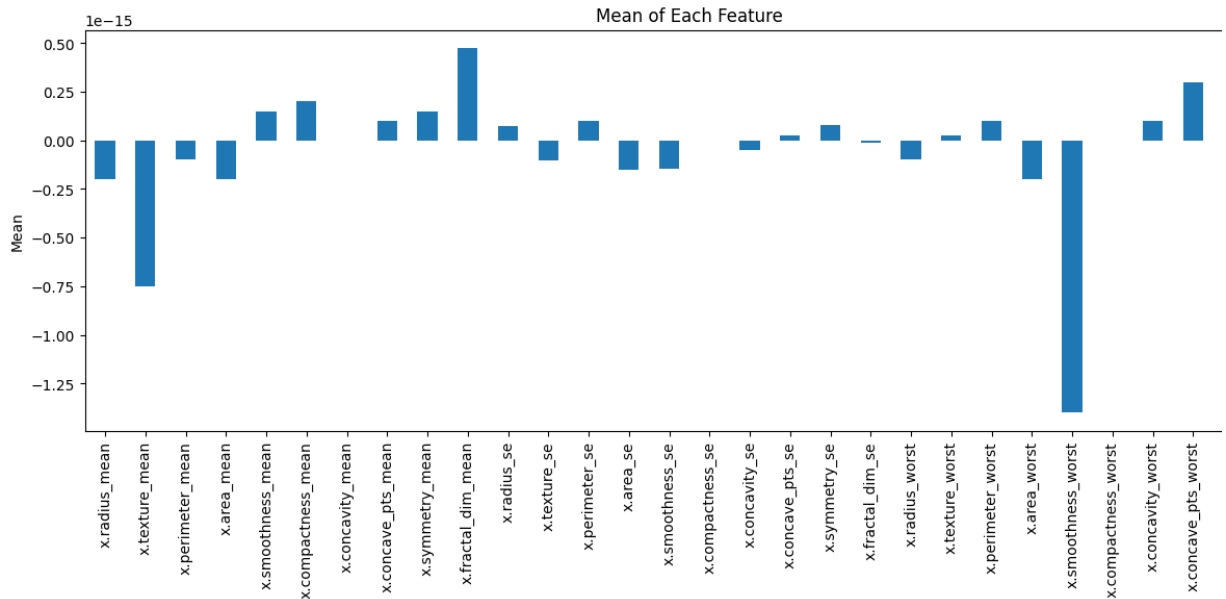
	x.radius_worst	x.texture_worst	x.perimeter_worst	x.area_worst	\
count	5.690000e+02	5.690000e+02	5.690000e+02	5.690000e+02	
mean	-9.990056e-17	2.497514e-17	9.990056e-17	-1.998011e-16	
std	1.000880e+00	1.000880e+00	1.000880e+00	1.000880e+00	
min	-1.726901e+00	-2.223994e+00	-1.693361e+00	-1.222423e+00	
25%	-6.749213e-01	-7.486293e-01	-6.895783e-01	-6.421359e-01	
50%	-2.690395e-01	-4.351564e-02	-2.859802e-01	-3.411812e-01	
75%	5.220158e-01	6.583411e-01	5.402790e-01	3.575891e-01	
max	4.094189e+00	3.885905e+00	4.287337e+00	5.930172e+00	

	x.smoothness_worst	x.compactness_worst	x.concavity_worst	\
count	5.690000e+02	569.000000	5.690000e+02	
mean	-1.398608e-15	0.000000	9.990056e-17	
std	1.000880e+00	1.000880	1.000880e+00	
min	-2.682695e+00	-1.443878	-1.305831e+00	
25%	-6.912304e-01	-0.681083	-7.565142e-01	
50%	-4.684277e-02	-0.269501	-2.182321e-01	
75%	5.975448e-01	0.539669	5.311411e-01	
max	3.955374e+00	5.112877	4.700669e+00	

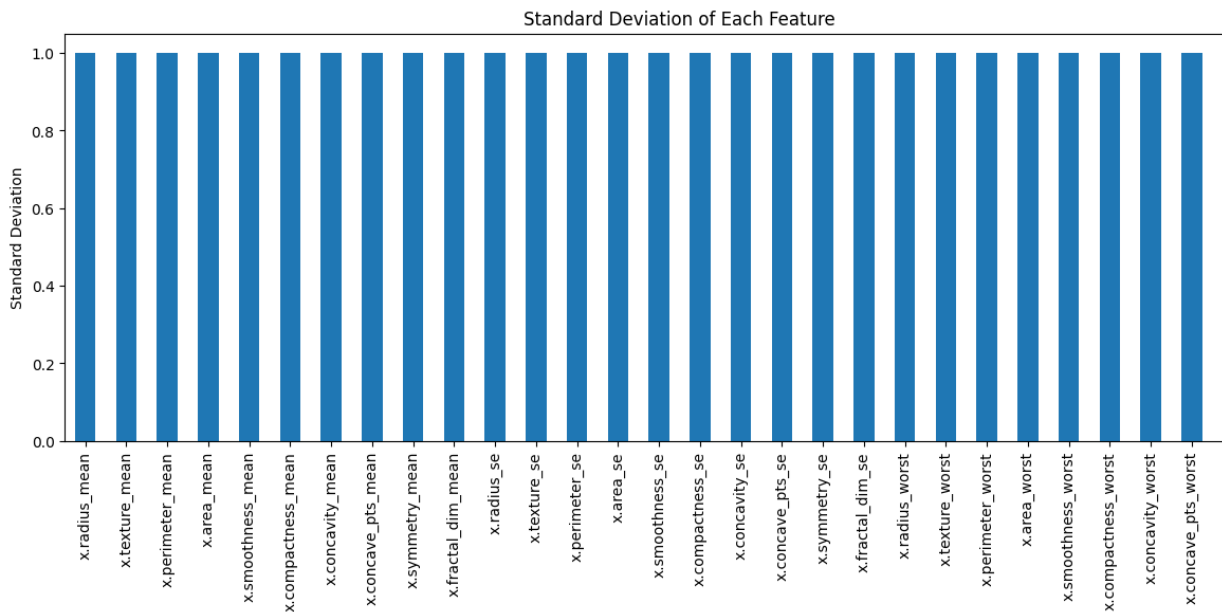
	x.concave_pts_worst	x.symmetry_worst	x.fractal_dim_worst
count	5.690000e+02	5.690000e+02	5.690000e+02
mean	2.997017e-16	2.497514e-16	1.998011e-16
std	1.000880e+00	1.000880e+00	1.000880e+00
min	-1.745063e+00	-2.160960e+00	-1.601839e+00
25%	-7.563999e-01	-6.418637e-01	-6.919118e-01
50%	-2.234689e-01	-1.274095e-01	-2.164441e-01
75%	7.125100e-01	4.501382e-01	4.507624e-01
max	2.685877e+00	6.046041e+00	6.846856e+00

[8 rows x 30 columns]

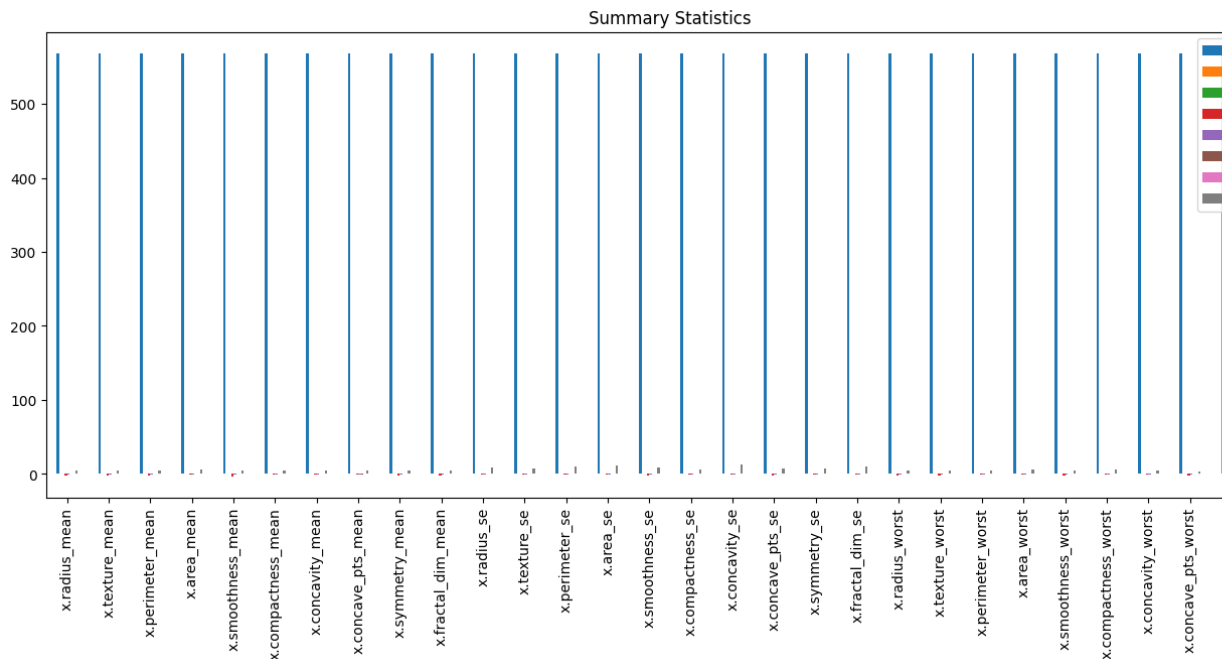
```
In [127... display_plots("mean", "bar", "Mean of Each Feature", "Mean")
```



```
In [128... display_plots("std", "bar", "Standard Deviation of Each Feature", "Standard Deviation")
```



```
In [129... summary.T.plot(kind="bar", figsize=(16,6))
plt.title("Summary Statistics")
plt.xticks(rotation=90)
plt.show()
```



Creating a model

```
In [132...] X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

```
In [133...] scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
In [134...] scaler.fit_transform(X)
```

```
Out[134...] array([[ -0.16679919, -1.1471623 , -0.18572799, ...,  0.21612292,
         0.12334653, -0.62929189],
        [ -0.29744572, -0.83300824, -0.26110605, ..., -0.63610973,
         0.45822712, -0.11724974],
        [ -1.31308049, -1.59395919, -1.30280622, ..., -0.7969026 ,
        -0.72922385, -0.34445459],
        ...,
        [  1.70485436,  2.08513394,  1.61593137, ...,  0.73382724,
        -0.53185462, -0.97397828],
        [  0.70228425,  2.0455738 ,  0.67267578, ...,  0.41406869,
        -1.10454895, -0.31840916],
        [  1.83834103,  2.33645719,  1.98252415, ...,  2.28998549,
         1.91908301,  2.21963528]], shape=(569, 30))
```

```
In [135...] model = LogisticRegression()

model.fit(X_train, y_train)
```

Out[135]:

- ▼ LogisticRegression

Parameters

	<code>penalty</code>	'deprecated'
	<code>C</code>	1.0
	<code>l1_ratio</code>	0.0
	<code>dual</code>	False
	<code>tol</code>	0.0001
	<code>fit_intercept</code>	True
	<code>intercept_scaling</code>	1
	<code>class_weight</code>	None
	<code>random_state</code>	None
	<code>solver</code>	'lbfgs'
	<code>max_iter</code>	100
	<code>verbose</code>	0
	<code>warm_start</code>	False
	<code>n_jobs</code>	None

Fitted attributes

Name	Type	Value
<code>classes_</code>	<code>ndarray[int64](2,)</code>	<code>[0,1]</code>
<code>coef_</code>	<code>ndarray[float64](1,30)</code>	<code>[[-0.48,-0.36,-0.46,...,-0.88,-0.69,-(</code>
<code>intercept_</code>	<code>ndarray[float64](1,)</code>	<code>[0.4]</code>
<code>n_features_in_</code>	<code>int</code>	<code>30</code>
<code>n_iter_</code>	<code>ndarray[int32](1,)</code>	<code>[18]</code>

```
In [136... y_pred = model.predict(X_test)
```

```
In [137... accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)
```

Accuracy: 0.9649122807017544

```
In [138... print(confusion matrix(y_test, y_pred))
```

$$\begin{bmatrix} 41 & 2 \\ 2 & 69 \end{bmatrix}$$

```
In [139... print(classification_report(y_test, y_pred))
```

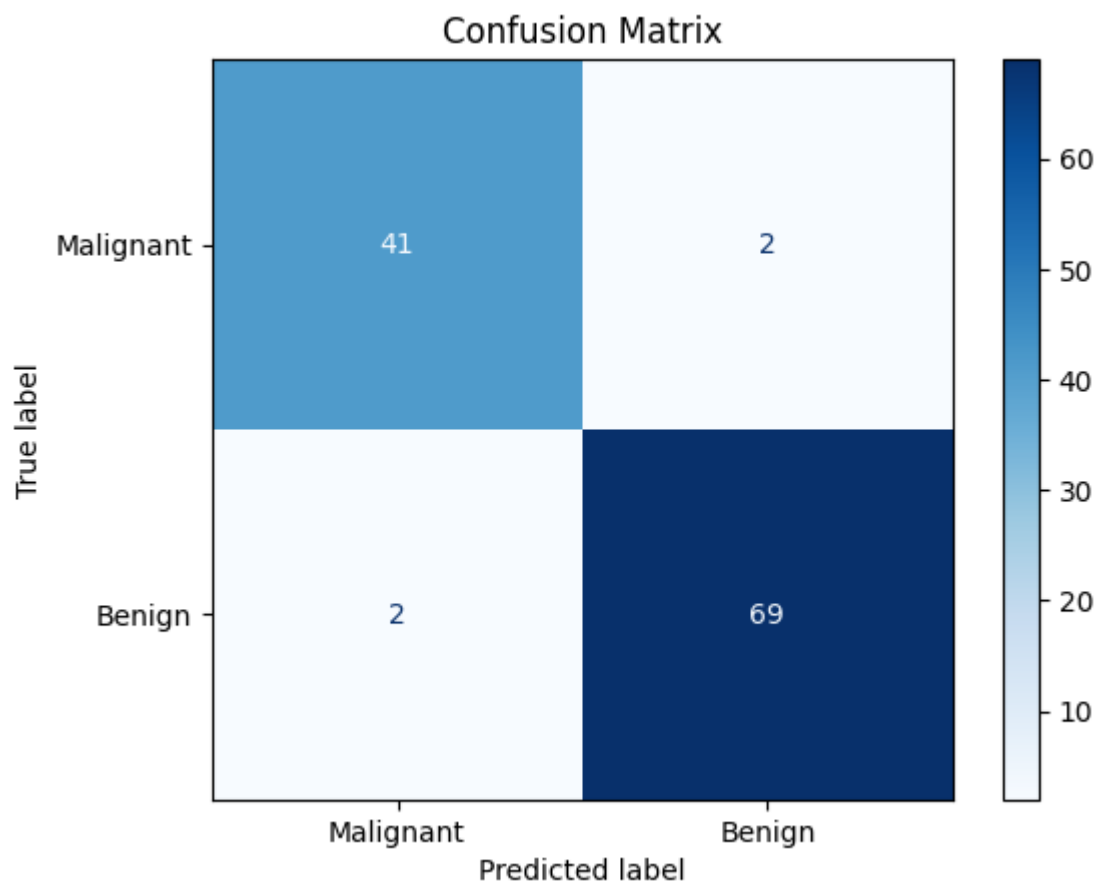
	precision	recall	f1-score	support
0	0.95	0.95	0.95	43
1	0.97	0.97	0.97	71
accuracy			0.96	114
macro avg	0.96	0.96	0.96	114
weighted avg	0.96	0.96	0.96	114

```
In [141]: # Create confusion matrix
cm = confusion_matrix(y_test, y_pred)

# Plot
disp = ConfusionMatrixDisplay(confusion_matrix=cm,
                              display_labels=["Malignant", "Benign"])

disp.plot(cmap="Blues")

plt.title("Confusion Matrix")
plt.show()
```



```
In [ ]:
```